

第八次实验报告

姓名： 林宏宇 学号： 23320093

1. 实验内容

存储器与控制器实验

实验概述： MIPS 架构 CPU 的传统流程可分为取指、译码、执行、访存、回写(Instruction Fetch, Decode, Execution, Memory Request, Write Back) 五阶段。完成了执行阶段的 ALU 部分，并进行了简单的访存实验，本实验将实现取指、译码两个阶段的功能。

4.1 实验要求

图 1 为本次实验所需完成内容的原理图，依据取指、译码阶段的需求，分别需要实现以下模块：

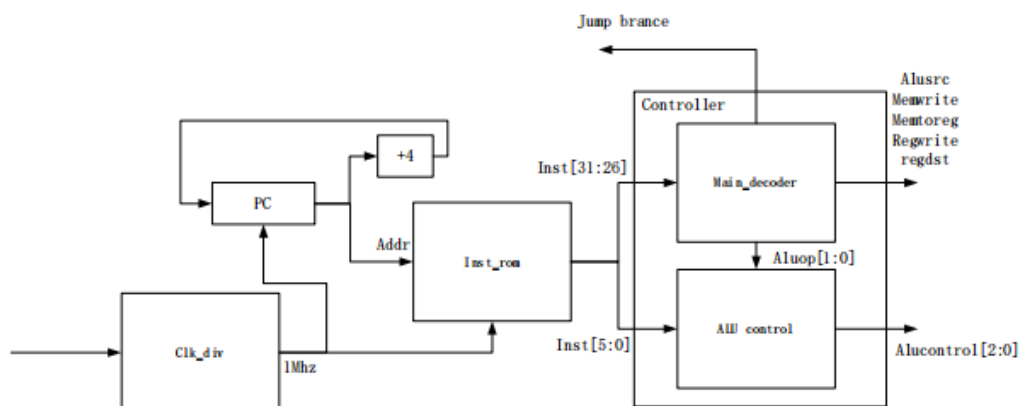


图 1

2. 实验过程

1. 创建工程，导入已经提供的模板：顶层模块 proc_top，控制器模块 controller (main_dec, alu_dec)，串口监视器模块 sys_monitor (串口模块 uart_top 模块 (rx、tx) 例化串口监视器模块需要的 ip 核；导入 coe 文件

2. 书写顶层模块 proc_top 代码之分频器 clk_div

```
// clk divide (时钟分频器) 50MHz -> 1MHz
reg [5:0] clk_divider; // 6-bit计数器, 最大值为 49

// 分频逻辑: 将50MHz时钟分频为1MHz
always @(posedge clk_i or negedge rst_n) begin
    if (!rst_n) begin
        clk_divider <= 6'd0; // 复位时清零
    end else if (clk_divider == 6'd49) begin
        clk_divider <= 6'd0; // 计数到 49 后清零
    end else begin
        clk_divider <= clk_divider + 1; // 计数器加一
    end
end

assign clk_run = (clk_divider == 6'd49); // 当计数器为 49 时输出 clk_run 高电平
```

3. 书写顶层模块之 pc 控制

```
reg inst_ce;
// PC 控制部分使用分频后的时钟信号
always @(posedge clk_run or negedge rst_n) begin
    if (!rst_n) begin
        pc <= 32'b0; // 复位时清零
        inst_ce <= 0;
    end else if (pc_clr) begin
        pc <= 32'b0; // 清零
    end else if (pc_run_en) begin
        pc <= pc + 4; // 连接addr
        inst_ce <= 1; // 连接rom的ena
    end
end
```

3. 书写顶层模块之 rom 例化以及创建 rom 的 ip 核

```

    // inst rom
    wire[31:0] rom_data;

    inst_rom u_inst_rom (
        .clka(clk_run),
        .ena(inst_ce),
        .addra(pc),
        .douta(rom_data)
    );

```

4. 书写顶层模块之控制器 controller 例化

```

    // controller
    controller u_controller(
        .op_i      (rom_data[31:26]),    // 指令的操作码字段 [31:26]
        .funct_i   (rom_data[5:0]),      // 指令的功能码字段 [5:0]
        .zero_i    (zero),               // 来自 ALU 的零标志
        .memtoreg_o (memtoreg),           // 是否从内存写寄存器
        .memwrite_o (memwrite),           // 是否写内存
        .pcsrc_o    (pcsrc),              // 分支指令跳转选择信号
        .alusrc_o   (alusrc),             // ALU 第二操作数选择
        .regdst_o   (regdst),             // 写寄存器目标选择
        .regwrite_o (regwrite),           // 是否写寄存器
        .jump_o     (jump),               // 跳转信号
        .alucontrol_o(alucontrol)         // ALU 控制信号
    );

```

4. 实现 controller 下的 main_dec, 按照控制器的译码信号表, 根据 op_i 定义输出信号

```

`timescale 1ns / 1ps

module main_dec(
    input wire [5:0] op_i,

    output wire memtoreg_o,
    output wire memwrite_o,
    output wire branch_o,
    output wire alusrc_o,
    output wire regdst_o,
    output wire regwrite_o,
    output wire jump_o,
    output wire [1:0] aluop_o
);
//main用于判断指令类型, 生成相应的控制信号

// add your code here
// 输出信号定义
assign regwrite_o = (op_i == 6'b000000) || (op_i == 6'b100011) || (op_i == 6'b001000); // R-type, lw, addi
assign regdst_o = (op_i == 6'b000000); // 只有 R-type 指令使用 rd 作为写入目标
assign alusrc_o = (op_i == 6'b100011) || (op_i == 6'b101011) || (op_i == 6'b001000); // lw, sw, addi
assign branch_o = (op_i == 6'b000100); // 只有 beq 指令是分支指令
assign memwrite_o = (op_i == 6'b101011); // 只有 sw 指令需要写入内存
assign memtoreg_o = (op_i == 6'b100011); // 只有 lw 指令需要从内存取数据
assign jump_o = (op_i == 6'b000010); // 只有 j 指令需要跳转
assign aluop_o = (op_i == 6'b000000) ? 2'b10 : // R-type 指令
                (op_i == 6'b000100) ? 2'b01 : // beq 指令
                2'b00; // lw, sw, addi 指令

endmodule

```

5. 实现 controller 下的 alu_dec, 根据指令段 aluop 和 funct 生成控信号

```

`timescale 1ns / 1ps

module alu_dec(
    input wire [5:0] funct_i, // R-type 指令的 funct 字段
    input wire [1:0] aluop_i, // 主控制单元的 ALUOp 信号
    output reg [2:0] alucontrol_o // 输出 ALU 控制信号
);

// 根据 aluop 和 funct 生成 ALU 控制信号
always @(*) begin
    case (aluop_i)
        2'b00: alucontrol_o = 3'b010; // 加法 (用于 lw, sw)
        2'b01: alucontrol_o = 3'b110; // 减法 (用于 beq)
        2'b10: begin // R-type 指令, 根据 funct 决定 ALU 操作
            case (funct_i)
                6'b100000: alucontrol_o = 3'b010; // add
                6'b100010: alucontrol_o = 3'b110; // subtract
                6'b100100: alucontrol_o = 3'b000; // and
                6'b100101: alucontrol_o = 3'b001; // or
                6'b101010: alucontrol_o = 3'b111; // set-on-less-than (slt)
                default: alucontrol_o = 3'bxxx; // 未定义的 funct 值
            endcase
        end
        default: alucontrol_o = 3'bxxx; // 未定义的 ALUOp 值
    endcase
end

endmodule

```

6. 书写顶层模块之 ALU 例化，以及实现 ALU 模块，参照之前 ALU 的实验，进行调整即可，其中 N、C、V 等输出信号暂时不需要使用

```
// ALU
wire [31:0] alu_b;          // ALU 的第二个操作数
wire [31:0] alu_result;    // ALU 运算结果

// ALU 第二操作数选择
assign alu_b = (alusrc) ? rom_data[15:0] : rom_data[25:21];

alu u_alu (
    .A_i      (rom_data[20:16]),      // 输入操作数 A
    .B_i      (alu_b),                // 输入操作数 B
    .op_i      (alucontrol),          // 操作码，由 controller 模块生成
    .result_o  (alu_result),          // 输出运算结果
    .Z         (zero),                // 输出零标志
    .N         (),
    .C         (),
    .V         ()
);

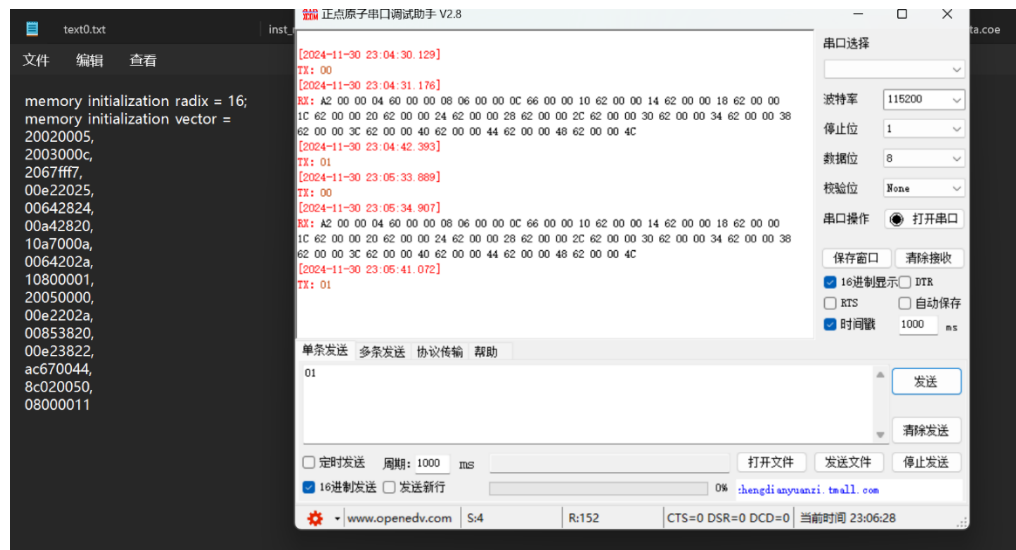
always @(*) begin
    // Initialize outputs
    result_o = 32'b0;
    Z = 1'b0;
    N = 1'b0;
    C = 1'b0;
    V = 1'b0;

    case (op_i)
        3'b010: begin // Addition
            {C, result_o} = A_i + B_i; // Capture carry out
            V = (A_i[31] & B_i[31] & ~result_o[31]) | (~A_i[31] & ~B_i[31] & result_o[31]); // Overflow
        end
        3'b110: begin // Subtraction
            {C, result_o} = A_i - B_i; // Capture carry out
            V = (A_i[31] & ~B_i[31] & ~result_o[31]) | (~A_i[31] & B_i[31] & result_o[31]); // Overflow
        end
        3'b000: begin // AND
            result_o = A_i & B_i;
        end
        3'b001: begin // OR
            result_o = A_i | B_i;
        end
        3'b101: begin // ~
            result_o = A_i | B_i;
        end
        3'b111: begin // SLT (Set Less Than)
            result_o = (A_i < B_i) ? 32'b1 : 32'b0;
        end
        default: begin // Default case: do nothing
            result_o = 32'b0;
        end
    end
endcase

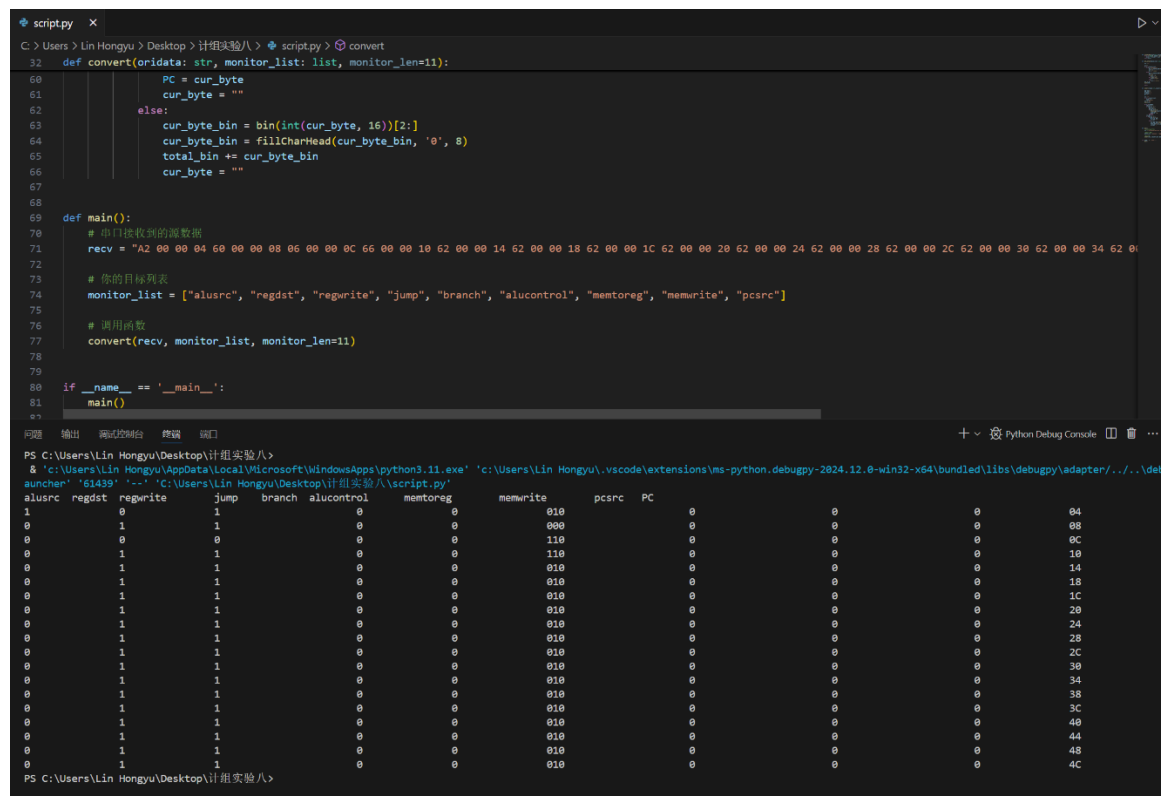
// Set flags
Z = (result_o == 32'b0); // Zero flag
N = result_o[31];        // Negative flag
end
```

3. 结果分析

将代码书写完，确认无误后进行烧录，并在串口调试助手上分别输入 00、01、00、01，检测功能正确性。当输入 00 时输出处理好的数据；输入 01 时复位停止不输出内容；再次输入 00 时又能输出数据。



将输出的数据，利用已有的 python 代码进行整理，直观地观察信号正确。



4. 总结

- 1.本次实验的完成难度不高，但是实验的逻辑要求和对基础知识的要求很高，需要综合课上所学知识，以及之前的实验积累
- 2.通过本次实验，加强了 MIPS 指令处理的理解。MIPS 架构 CPU 的传统流程可分为取指、译码、执行、访存、回写(Instruction Fetch, Decode, Execution, Memory Request, Write Back) 五阶段。本次实验注重取指和译码阶段，在代码模板和书写部分代码的过程可以加深对其理解。
- 3.同时结合指令处理流程图可以有利于加强逻辑思维，明确各个模块之间的关系，在我写代码的过程中发挥了强大作用